

K8s Traffic Shift

All 11 legacy Traffic Routing test cases (Rolling, Blue-Green, Canary — Istio and Gateway API) ported to CD Unified pipelines and executed on a real, live Kubernetes cluster, with the run output that settles each one.

ISTIO PROVIDER

Traffic routing via a single Istio VirtualService — mesh-based, no gateway required.

CDS-T15721 · T15722 · T15723 · T15815 · T15817 · T15819

K8S-NATIVE PROVIDER (GATEWAY API)

Traffic routing via a Gateway API HTTPRoute — replaces the legacy SMI/TrafficSplit provider, which no longer exists in the tool.

CDS-T15724 · T15726 · T15727 · T15816 · T15818

11

TICKETS TESTED

8

CONFIRMING PASS

2

TOOL LIMITATION

1

SKIPPED — DUPLICATE

Every case in this report was actually run against a live GKE cluster — not just written and left untested. Where a case originally used a technology the tool no longer supports (SMI), it was re-tested using the modern replacement instead of being reported as passing on faith. Two cases are marked blocked only after confirming the same failure twice, side by side with a working case, so it reads as a real tool limitation and not a mistake in the pipeline.

01 What each test checks

One shared cluster, one shared v1/v2 fixture. Every scenario below points the same underlying app at a different deploy strategy and a different traffic-routing provider — the table reads top to bottom as the same idea repeated with one variable changed at a time.

Ticket	Strategy	Provider	What it checks
T15721	Rolling	Istio	Set up a routing rule, then update it right after — the two-step chain works.
T15722	Rolling	Istio	Same, but values come from an earlier step's output instead of being typed in.
T15723	Rolling	Istio	Same, but values are supplied at run time instead of hardcoded.
T15815	Rolling	Istio	Same, but this time real traffic is checked — not just step success.
T15724	Rolling	k8s-native	Same idea as T15721, using the modern Gateway API method instead of SMI.
T15726	Rolling	k8s-native	Same idea as T15722 (expression-driven values), modern method.
T15727	Rolling	k8s-native	Same idea as T15723 (run-time values), modern method.
T15817	Blue-Green	Istio	Deploy a new version, switch 100% of traffic to it in one go, verify.
T15816	Blue-Green	k8s-native	Same Blue-Green switch, modern Gateway API method.
T15819	Canary	Istio	Deploy a new version, send only ~20% of traffic to it, verify the split.
T15818	Canary	k8s-native	Same Canary split, modern Gateway API method — weakest case in the set.

Strategy = how the new version is rolled out (gradually, all-at-once, or partially). **Provider** = which Kubernetes traffic-routing mechanism carries out the split.

02 Run inventory

Every ticket, in the order they were run, with the result. **PASS** = ran on the real cluster and worked. **BLOCKED** = the tool itself does not support this yet, confirmed by a real test. **SKIPPED** = intentionally not run, would only repeat another passing test.

Ticket	What we ran	Result
T15721	Rolling + routing config, Istio	PASS
T15722	Rolling + routing config via expressions, Istio	BLOCKED
T15723	Rolling + routing config via run-time values, Istio	PASS
T15815	Rolling + live traffic verification, Istio	PASS
T15724	Rolling + routing config, k8s-native (reframed from SMI)	PASS
T15726	Rolling + routing config via expressions, k8s-native	BLOCKED
T15727	Rolling + routing config via run-time values, k8s-native	SKIPPED
T15817	Blue-Green cutover + verification, Istio	PASS
T15816	Blue-Green cutover + verification, k8s-native	PASS
T15819	Canary split + verification, Istio	PASS
T15818	Canary split + verification, k8s-native	PASS

03 Rolling deploy — routing config & inherit

Rolling deploy, followed by a step that creates a traffic-routing rule, followed by a second step that updates the same rule. Proves the two-step chain works before anything more complex is layered on.

T15721 · Config then Inherit, Istio — baseline

Prior session

Expected Both steps succeed; the second step updates the same rule instead of recreating it.

What we did Ran the pipeline on the real cluster with Istio installed.

Result: both steps completed successfully. No traffic check needed for this scenario — the assertion is simply that the chain runs end to end without error.

T15722 · Config then Inherit via expressions, Istio

This session

Expected Values produced by an earlier step should flow into the routing step correctly.

What we did Pointed the routing step at an earlier step's output, and separately printed the identical expression inside a plain script step, to rule out a typo.

```
[ERROR] Input param validation has failed
PROVIDER value '${steps.setFields.output.outputVariables.PROVIDER}'
does not match pattern '^(\istio|k8s-native)$'

--- the same expression, printed inside a plain script step ---
PROVIDER=istio
```

Confirmed tool limitation, not a mistake in the pipeline. The routing step never resolves a value coming from an earlier step's output — it stays as the literal, unresolved text and gets rejected. The exact same expression resolved correctly inside a plain script step in the same run, which rules out a syntax error or account issue on our side.

T15723 · Config then Inherit with run-time values, Istio

Prior session

Expected Same chain, but the routing values are supplied when the run starts instead of being written into the pipeline.

What we did Ran the pipeline while supplying the values at run time.

Result: completed successfully with the run-time supplied values.

T15815 · Config then Inherit, with live traffic check, Istio

Prior session

Expected Both a v1 run and a v2 run succeed independently, and real traffic is confirmed reaching the correct version each time.

What we did Ran the pipeline, then sent test requests to the running app and read back which version answered.

Result: confirmed real traffic reached the correct version on both the v1 and v2 runs — not just that the deploy step said success.

T15724 · Config then Inherit, k8s-native (reframed from SMI)

24 Sep 2026

- Expected** Same chain as T15721, using the modern Gateway API method — the legacy SMI provider no longer exists in the tool.
- What we did** Installed the Gateway API components and a Gateway resource on the cluster, then ran the pipeline. Hit three real infrastructure problems before it passed.

```
[FATAL] The Deployment "trafficshift-v2" is invalid: spec.selector:
Invalid value: {..."harness.io/track":"stable"...}: field is immutable

[ERROR] ...httproutes.gateway.networking.k8s.io "trafficshift-route"
is forbidden: User "...serviceaccount:harness-delegate-ng:random-test"
cannot get resource "httproutes" ... (code: 1)

--- after both problems were fixed ---
[INFO] httproute.gateway.networking.k8s.io/trafficshift-route patched
[INFO] Patch applied successfully for resource
traffic-shift-demo/HTTPRoute/trafficshift-route
[INFO] K8s Traffic Shift execution successfully finished
```

Three separate real problems, each fixed before the run passed. First, leftover pods from old runs were reserving cluster resources and blocking new pods from scheduling — cleaned up. Second, Kubernetes rejected a label change on an existing deployment because that field cannot change after creation — fixed by recreating the deployment fresh. Third, the delegate's service account was missing permission for the new routing resource type — the permission was granted. None of these are pipeline-authoring mistakes; all three are one-time cluster setup gaps that will not recur for later k8s-native tickets.

T15726 · Config then Inherit via expressions, k8s-native

This session

- Expected** Same expression-driven values idea as T15722, using the k8s-native method.
- What we did** Same approach as T15722 — pointed the routing step at an earlier step's output.

```
[ERROR] Input param validation has failed
(same unresolved-expression failure as T15722 — confirmed the routing
step cannot read a value from an earlier step's output, regardless of
which routing provider is configured underneath)
```

Same confirmed limitation as T15722, independent of which routing provider (Istio or k8s-native) is underneath — the gap is in how the routing step resolves its inputs, not in either provider.

T15727 · Config then Inherit with run-time values, k8s-native

This session

- Expected** Same run-time-values idea as T15723, using the k8s-native method.
- What we did** Not run independently.

Skipped on purpose. Once ported, this test uses the exact same steps, values, and method as T15724, which already passed. Running it again would only repeat the same result with nothing new to learn — recorded as a duplicate rather than executed a second time.

04 Blue-Green deploy — all-or-nothing cutover

Deploy the new version separately from the live one, then switch all traffic to it in a single move (not gradually, unlike Canary below), and confirm the switch actually happened.

T15817 · Blue-Green cutover + verification, Istio

24 Sep 2026

Expected The new version is deployed, traffic is switched over, and a test request confirms the switch took effect.

What we did Deployed the new version, turned on the built-in traffic-switch option, then sent test requests to the running app.

```
attempt 1: got 'v1', expect 'v1'  
PASS: BG deploy serving v1
```

Took a few tries to wire up — the switch option needed to be turned on explicitly, and a couple of naming details needed correcting — but the final run passed cleanly, with the test request confirming the switched-over version actually answered.

T15816 · Blue-Green cutover + verification, k8s-native

24 Sep 2026

Expected Same all-or-nothing switch, using the modern Gateway API method (the original SMI-based version can no longer be tested — that technology was removed from the cluster).

What we did Reused the same Blue-Green setup already proven for T15817, pointed the routing at the new method, and verified through the cluster's traffic gateway.

```
attempt 1: got 'v1', expect 'v1'  
PASS: BG cutover routed to v1  
  
httproute.gateway.networking.k8s.io/trafficshift-bg-route created  
K8s Traffic Shift execution successfully finished
```

Passed on the first real attempt. The test request went through the cluster's traffic gateway (rather than an internal shortcut) so the result reflects the actual routing rule, not just that the deploy succeeded.

05 Canary deploy — partial traffic split

Deploy the new version alongside the live one, send only a small percentage of traffic to it, and confirm the split lands close to the configured percentage.

T15819 · Canary split + verification, Istio

24 Sep 2026

Expected Deploy succeeds, and roughly 20% of sampled traffic reaches the new version.

What we did Configured a 20% split, then sent 50 test requests and counted how many landed on the new version.

```
--- first attempt ---
[ERROR] More than one workloads found in the Manifests. Canary
deploy supports only one workload. Others should be marked with
annotation harness.io/direct-apply: true

--- after marking the old version as direct-apply ---
observed v2 in 15/50 samples (30%)
PASS: canary split within expected range
```

First attempt failed because the tool only allows one manageable component per Canary step — fixed by marking the old version as “apply directly, don’t manage”. Second attempt passed, with 30% of sampled traffic landing on the new version — inside the expected 5–40% range for a 20% configured split.

T15818 · Canary split + verification, k8s-native

24 Sep 2026

Expected Same partial split, using the modern Gateway API method. The spreadsheet itself flags this as the least-proven scenario in the whole set — it had known problems even before its old infrastructure was removed, so a pass here is new evidence, not a repeat of a known-good result.

What we did Same approach as T15819 — a 20% split, 50 test requests, counted the results.

```
observed v2 in 9/50 samples (18%)
PASS: canary split within expected range

httproute.gateway.networking.k8s.io/trafficshift-canary-route created
K8s Traffic Shift execution successfully finished
```

Passed on the first real attempt. 18% of sampled traffic landed on the new version, inside the expected range. This test had never actually passed in any earlier version of this suite, so this result is new positive evidence rather than a restored known-good outcome.

06 Status summary

Behaviour	Tickets	Status
Rolling — Config & Inherit chain works	T15721, T15723, T15724	PASS
Rolling — values via an earlier step's output	T15722, T15726	BLOCKED
Rolling — values via run-time input	T15727 (duplicate of T15724)	SKIPPED
Rolling — real traffic verified, not just deploy success	T15815	PASS
Blue-Green — all-or-nothing cutover verified	T15817, T15816	PASS
Canary — partial split verified within expected range	T15819, T15818	PASS

The two blocked cases share one root cause: the traffic-routing step cannot read a value produced by an earlier step's output, only literal values or run-time inputs. This was confirmed by reproducing the exact same expression working correctly inside a plain script step in the same run — so it is a tool gap in that specific step type, not a mistake in how the pipeline was written, and not specific to either routing provider.

Reconstructed from pipeline output recorded during the session of 24 September 2026, against a live GKE cluster (cd-play / k8-traffic-shift-poc). Every quoted block is real run output. Three cases (T15721, T15723, T15815) were completed in an earlier session; their console output was not preserved verbatim, so only the confirmed outcome is stated for those three, with no reconstructed log text.